CHAOS OS

Team roles and individual pitches

Internal reference for our four-person team. Replace "Member 1-4" with names.

How we divided the work, fairly

- **Equal weight:** four roles of similar size, each with a clear part of the system, a demo moment and a 2-minute speaking slot.
- **Everyone owns something real:** each role maps to code and results you can open and explain, not just slides.
- **Nobody is a single point of failure:** every member also learns the shared basics below and can answer simple questions about any part.
- **Honest credit:** we say "we built" as a team. If asked about tools, we say plainly that we used AI assistants (Claude for development help, gpt-4o-mini inside the project), and we only present what we understand and can explain ourselves.
- **Swap if it fits better:** match roles to interests (ML-minded person takes Member 2, ops-minded takes Member 4). Fairness matters more than the default order.

Member	Role	Speaks	Demo
Member 1	Architect and Measurement Lead	Opens the presentation	Dashboard Overview and the Matrix page
Member 2	AI and Machine Learning Lead	Second speaker	Policies page, Learning page, and one LLM agent trial on the Incidents page with its tool trail.
Member 3	Reliability (SRE) and Safety Lead	Third speaker	On-call page
Member 4	DevOps, Security and Delivery Lead	Fourth speaker	Jenkins

What everyone must know (shared basics)

- The 30-second pitch (pitch guide, section 1) and the real-world problem: long outages because the fix is a guess (MTTR).
- The architecture in one sentence: a web shop (API, Redis, Postgres) behind Toxiproxy, watched by Prometheus, fixed by policies through a safety validator and Ansible, gated by Jenkins.
- The five headline numbers: no universal fix; bandit 88% vs runbook 33%; 18/21 surprise failures healed; 0 unsafe actions; the gate blocks a one-character bug.
- The limitations: one laptop, small samples, the compound failure is unsolved.
- How to start it: start.bat (local) or share.bat (online link, login admin + password from deploy/secrets.yml).

Presentation running order (about 10 minutes + Q&A)

Time	Who	What
0:00	Member 1	Problem, architecture, measurement, result 1
2:00	Member 2	Responders, bandit, LLM ablation
4:00	Member 3	Safety, SLOs, on-call agent + live Break-it demo
7:00	Member 4	Pipeline, gate demo (green vs red), security, deployment, summary
9:30	All	Q&A: the owner answers first, anyone may add

Member 1: Architect and Measurement Lead

Role	Details
Owns	The problem statement, the architecture (11 services), how faults are injected, and how recovery is measured.
Speaking slot	Opens the presentation (2 min) and hands over to Member 2.
Demo moment	Dashboard Overview and the Matrix page: every failure x every fix.
Study	Study guide chapters 1, 3, 4 · controller/chaosctl/analysis.py, experiment.py, scenarios.py · docker-compose.yml

YOUR PITCH (ABOUT 45 SECONDS)

"Every system fails; the question is how fast it recovers. We built a lab of 11 containers: a web shop with a Redis cache and a Postgres database, and Toxiproxy in between so we can inject real failures such as a dead cache or a slow network. A load generator sends 20 requests a second and times every single one, and we call the service recovered only after 5 straight healthy seconds at 99% success and under 300 milliseconds. Our first finding: there is no universal fix. Each failure has its own best action, and a wrong one costs up to 1,128 failed requests instead of about 30."

Your STAR story

STAR	Your part
Situation	Systems fail and nobody measures whether runbooks work.
Task	Build a fair, precise way to inject failures and measure recovery.
Action	Docker Compose lab, Toxiproxy and docker-kill fault injection, open-model load generator, unit-tested recovery definition.
Result	Sub-second recovery measurement; proved each of 5 failures needs a different fix.

Questions you answer first

- How exactly do you define recovered? (model answers: study guide chapter 10)
- Why not measure recovery with Prometheus? (model answers: study guide chapter 10)
- Why does restarting Redis not fix Redis latency? (model answers: study guide chapter 10)
- Why Docker Compose and not Kubernetes? (model answers: study guide chapter 10)

YOUR HAND-OFF LINE

"So we know which fix is right. But can a machine learn that by itself? Member 2."

Member 2: AI and Machine Learning Lead

Role	Details
Owns	The five responders: runbook, expert rules, the LinUCB bandit, gpt-4o-mini and the LLM agent with tools; the learning curve and the 56-trial ablation.
Speaking slot	Second speaker (2 min).
Demo moment	Policies page, Learning page, and one LLM agent trial on the Incidents page with its tool trail.
Study	Study guide chapters 5, 9.1-9.4 · controller/chaosctl/bandit.py, policies.py, agent.py, learning.py

YOUR PITCH (ABOUT 45 SECONDS)

"We compared five ways of deciding the fix. A classic runbook, 'restart whatever looks broken', was right only 33% of the time. A contextual bandit, a small learning algorithm, learned from our experiment logs and reached 88% with no hand-written rules, and it even solved a failure it had never seen, where the runbook failed every time. We also tested gpt-4o-mini honestly with a 2 x 2 experiment: one paragraph of advice cut a 50-second recovery to 3 seconds but then made it skip a needed restart; giving it tools alone only made it slower; tools plus advice was the most reliable. Lesson: test AI prompts like code."

Your STAR story

STAR	Your part
Situation	Expert rules work but someone must study every new failure; LLM agents are hyped but untested.
Task	Find a responder that learns the right fix, and test what really helps an LLM.
Action	LinUCB bandit on 8 symptom features with a speed-minus-safety-cost reward; tool-using LLM agent; 2 x 2 ablation over 56 trials.
Result	Bandit 88% vs runbook 33%, 3/3 on an unseen failure; clear, honest LLM findings.

Questions you answer first

- Isn't the bandit memorising failure names? (model answers: study guide chapter 10)
- Why a bandit and not deep learning? (model answers: study guide chapter 10)
- Why gpt-4o-mini? (model answers: study guide chapter 10)
- Isn't the SRE advice just giving the LLM the answer? (model answers: study guide chapter 10)

YOUR HAND-OFF LINE

"Smart decisions are dangerous without guardrails, and real failures are not announced. Member 3."

Member 3: Reliability (SRE) and Safety Lead

Role	Details
Owns	The safety validator and red team, SLOs and error budgets, burn-rate alerts, Alertmanager, and the on-call agent with human approval.
Speaking slot	Third speaker (2 min) and runs the live Break-it demo.
Demo moment	On-call page: Break it (surprise), watch the incident timeline, then postgres_down and Approve; Grafana SLO row.
Study	Study guide chapters 2.4-2.5, 4.6, 6 · safety-policy.yml · observability/prometheus/rules/slo.yml · controller/chaosctl/oncall.py, safety.py

YOUR PITCH (ABOUT 45 SECONDS)

"Nothing runs without passing our safety shield: approved environment, an allowlist, no repeated actions, at most three actions, and a human for risky ones. Our red-team policy tried drop_database; it was blocked, and zero unsafe actions ever ran. Then we made it work like production: simulated users measure an SLO of 99% good requests, Prometheus pages when the error budget burns 14 times too fast, and our on-call agent wakes up, confirms the problem, fixes it, and asks me before restarting the database. In 21 surprise failures it healed 18 on its own in about 25 seconds, and handed the 3 unfixable ones to a human instead of looping."

Your STAR story

STAR	Your part
Situation	Automation can make outages worse, and real failures are never announced.
Task	Make every action safe and heal unannounced failures like a human on-call engineer.
Action	YAML safety policy + Ansible allowlist + red team; SLI/SLO with multi-window burn-rate alerts (unit-tested); Alertmanager webhook; agent that confirms, fixes, asks for approval, escalates, writes post-mortems.
Result	0 unsafe actions; 18/21 surprise failures healed; median 11.8 s to alert, 25.8 s to healthy.

Questions you answer first

- What stops the AI from doing damage? (model answers: study guide chapter 10)
- What is an error budget and why 14.4? (model answers: study guide chapter 10)
- What if nobody approves? (model answers: study guide chapter 10)
- How do you know the agent did not cheat? (model answers: study guide chapter 10)

YOUR HAND-OFF LINE

"All of this has to ship safely, every time. Member 4."

Member 4: DevOps, Security and Delivery Lead

Role	Details
Owns	The Jenkins pipeline (10 stages), the resilience gate and demo bug, security scanning, Ansible, deployment (HTTPS, login, tunnel), and the start/share scripts.
Speaking slot	Fourth speaker (2 min), closes, and leads the Q&A hand-offs.
Demo moment	Jenkins: main green vs demo branch red; the security report; share.bat public link.
Study	Study guide chapters 7, 8 · Jenkinsfile · resilience-gate.yml · scripts/security-scan.sh · deploy/

YOUR PITCH (ABOUT 45 SECONDS)

"Every change goes through a 10-stage Jenkins pipeline: build, 45 unit tests, tests for our alert rules, a security scan for leaked secrets and vulnerable packages, a real deployment, then a resilience gate that injects six real failures and a surprise on-call drill. Our demo branch has a one-character bug that passes unit and smoke tests, and only the resilience gate blocks it. The first security scan found real problems, and we fixed them: upgraded packages, seven containers made non-root, 61 vulnerabilities removed from one image. And it deploys anywhere with HTTPS and a login in one command, or shares from a laptop with one click."

Your STAR story

STAR	Your part
Situation	Pipelines test code on a healthy system but never test recovery or security.
Task	Make resilience, alerting, security and on-call behaviour release gates.
Action	Jenkins multibranch job as code with isolated labs; gitleaks + Trivy + SBOMs; resilience gate; on-call drill; Caddy HTTPS deploy via Ansible; start/share scripts.
Result	Main passes in under 6 minutes; the one-character bug is blocked; 0 secrets, 61 to 0 CVEs in the frontend image.

Questions you answer first

- Walk me through the pipeline. (model answers: study guide chapter 10)
- What does the gate catch that tests miss? (model answers: study guide chapter 10)
- What did the security scan find? (model answers: study guide chapter 10)
- How is it deployed and secured? (model answers: study guide chapter 10)

YOUR HAND-OFF LINE

"To sum up: we measure, learn, act safely, and block releases that cannot recover. Questions?"

Rehearsal checklist

- Run start.bat 10 minutes before; do one warm-up Break-it.
- Each member says their pitch out loud three times, under 60 seconds.
- Swap test: each member answers one question from another role.
- Backup plan: if the live demo fails, show the screenshots in the study guide.
- Fill names and college details in the paper's author block and the study guide cover.